

AI_tour_guide.ae

Smart UAE AI Tourism Assistant

A web application that turns four trip settings into a full day-by-day United Arab Emirates itinerary — priced in dirhams, refinable by chat, plotted on a map and exported as a PDF.

ACTIVITY TITLE AI_tour_guide.ae — Smart UAE AI Tourism Assistant

STUDENT NAME Ashwanth Vijay

GRADE & SECTION Grade 9 – C

ROLL NUMBER 9

SUBJECT Artificial Intelligence

ACADEMIC YEAR 2026 – 2027



LIVE WEBSITE

ai-tour-guide-ae.netlify.app

The working app — open the studio and generate a plan.



SOURCE CODE

github.com/ashwanthvijay1234-debug/AI_tour_guide.ae

Every file in this report, and the full commit history.

HTML

CSS

JavaScript

Google Gemini API

jsPDF

Google Maps

Aim and Concept

What the project sets out to do, the problem it answers, and why that matters here.

Aim

To build a working Artificial Intelligence web application that solves a real problem for visitors to the United Arab Emirates, and to show how **generative AI** and **Natural Language Processing** can be applied to tourism — not as a chatbot bolted onto a page, but as the engine that produces the product itself.

Chosen Topic

Smart tourism. A generative UAE travel planner that writes a full day-by-day itinerary from four settings — destination, trip length, budget and interests — then lets the user refine it by talking to it in plain English.

Problem Statement

A printed guidebook cannot know who is reading it.

NOT PERSONAL

A student on AED 2,000 and a honeymoon couple get the same 300 pages, and must filter it themselves.

NOT FLEXIBLE

A 44 °C afternoon or a tired toddler rewrites the day; the printed page cannot move to meet it.

NOT CURRENT

Prices, opening hours and whole attractions change faster than editions are reprinted.

NOT EFFICIENT

Comparing maps, prices and timings across seven emirates takes an evening — and still over-drives.

AI Solution Proposed

A four-stage pipeline. The code for every stage is in the appendix.

#	Stage	What happens
01	Understanding preferences	The four settings become a natural-language brief. NLP reads what is <i>implied</i> as well as typed — “family” means shorter days and earlier evenings.
02	Prompt engineering	A system instruction fixes the role, the currency, the pacing rules and the exact JSON shape. Constraining the output turns a chatty model into a reliable component.
03	Generative reasoning	Gemini balances geography, timing and money token by token, writing a plan that did not exist before. Nothing is retrieved from a list of stored tours.
04	Structured rendering	The reply is parsed back into data, drawn on screen, typeset into a PDF and plotted on a map.

Two modes, and the app always says which is running. With no API key an offline planner built into the JavaScript writes the itinerary, so the project works with no internet at all; with a key, Google Gemini writes it. A badge on every plan names the engine that produced it, so “*is the AI actually running?*” can always be answered.

Importance to UAE

- **Economy** — tourism is a pillar of growth beyond oil, and a country that is easier to plan a trip to is easier to choose.
- **Sustainability** — grouping each day's stops by area cuts driving, and with it fuel, emissions and time lost in traffic.
- **Culture** — the model must include Emirati heritage in every plan, not only malls and towers.
- **Access** — LLMs already work in Arabic, Hindi, Mandarin and Russian, so no separate edition need be printed.

This serves **UAE Vision 2071** and the **National AI Strategy 2031**, which names tourism among the sectors AI should serve.

Webpage Design Plan

The rough sketch drawn before any code was written, and the file structure it turned into.

Layout Sketch

Paste the hand-drawn layout sketch here

A rough sketch of the two pages is enough — boxes and labels, not a finished design.

What the sketch plans

- **Page 1 — landing.** Header, hero, a mock of the app, then numbered sections and a footer.
- **Page 2 — the studio.** An app bar over three columns: trip setup, AI chat, and the output tabs.
- **Output tabs.** Itinerary, PDF and Map share one panel area.
- **Responsive.** The three columns stack into one below 1040px.

The files it became

index.html	landing page
studio.html	the application
css/tokens.css	colours, fonts, spacing
css/base.css	reset + shared controls
css/index.css	landing-page layout
css/studio.css	app layout
js/index.js	header, scroll, reveals
js/studio.js	planner, AI, chat, map, PDF

Screenshots

Paste each screenshot into the frame below it. The captions are already written.

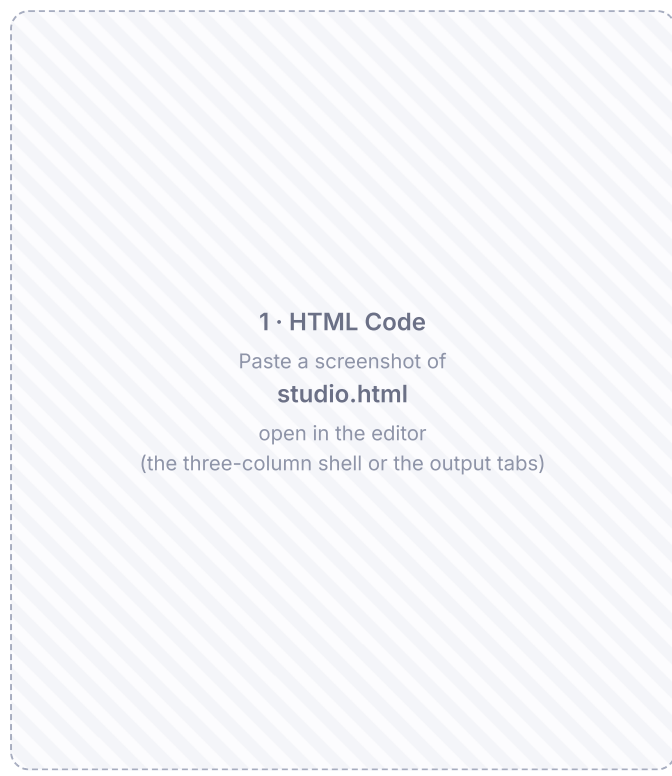


Fig. 1 — The HTML for the studio: an ARIA tablist for the Itinerary, PDF and Map panels, with the trip-setup form on the left.

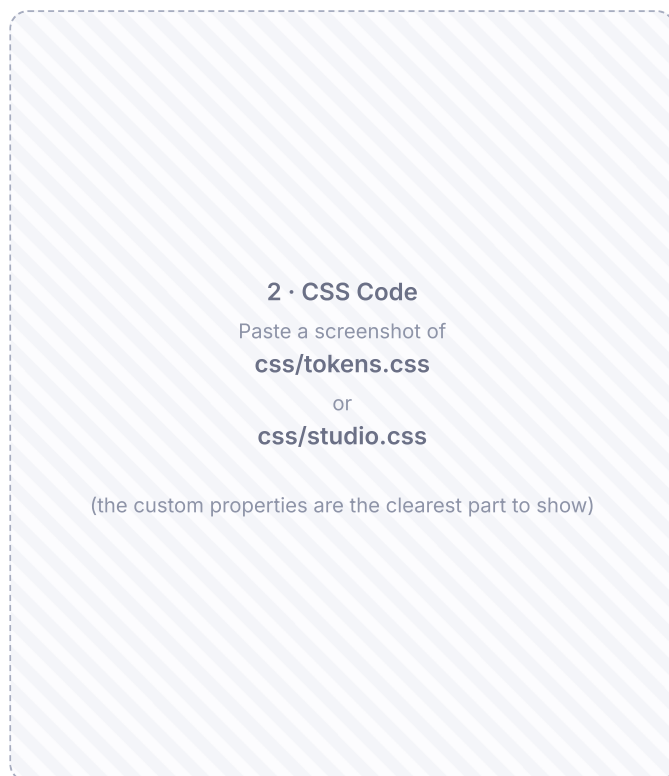


Fig. 2 — The design system in CSS custom properties — one source of truth for colour, type and spacing across both pages.



Fig. 3 — The landing page: an animated CSS sky layered over the Dubai skyline, explaining the project before the user opens the studio.



Fig. 4 — The final output: a complete day-by-day itinerary with time windows, reasons and dirham costs, ready to export as a PDF.

Reflection

What the build taught me, what went wrong on the way, and what I would take to the next project.

What I Learned

- **A language model becomes useful when you constrain it.** Asked an open question it writes pleasant paragraphs. Pinned to a role, a currency, a pacing rule and an exact JSON shape, it becomes a component I can build a product on. Prompt engineering turned out to be the real engineering in this project.
- **Never trust a model's output format.** Even when told "JSON only, no markdown fences", it sometimes returns fences or a sentence first. The parser strips fences, retries on the widest {...} substring, and renders prose safely rather than failing.
- **Never trust model output as markup either.** Every element is built with createElement and textContent, never innerHTML, so a stop title containing a script tag becomes a badly-named stop instead of running code.
- **Honesty has to be designed in.** Both modes produce something that looks like an itinerary, so I added a provenance badge and a Test key button that makes one real request and reports the failure instead of hiding it.

Challenges Faced

Challenge	How I solved it
Model IDs kept expiring	I had hard-coded model names. Google retired them, every call returned 404, and the app silently fell back to the offline planner — with a valid key in the box. Now the app asks the API which models the key can really call and filters its preference list against that.
Errors blamed the wrong model	The fallback chain reported the <i>last</i> error, so a bad key surfaced as "gemini-1.5-pro not found" — a model the user never picked. It now keeps the <i>first</i> error, from the model actually chosen.
The same plan every time	Two causes. The counter seeding the shuffle reset on every page load, so a reload gave a byte-identical plan; it now persists. And the place pool was smaller than the trip needed, so every place was used every run — the pool grew to 103 real UAE locations.
"Cheaper" froze the plan	Sorting by cost always picks the same cheapest stops. Adding a seeded jitter to the prices keeps the trip cheap while varying which cheap places are chosen.
Empty days on long trips	A 14-day trip needs 84 stops and the pool holds fewer, so later days printed blank. The planner now cycles the pool instead, which is what a real fortnight looks like anyway.

Skills Developed

TECHNICAL

Semantic HTML, CSS custom properties and responsive layout, modern JavaScript with async/await, fetch and the DOM API.

AI

Prompt engineering, constraining output to a schema, temperature, model fallback chains, and reading an API's error responses properly.

ENGINEERING

Debugging from symptoms, defensive coding, accessibility with ARIA, keeping secrets out of a repository, and version control with Git.

Conclusion

The project does what it set out to do: four answers become a complete, priced, mapped and printable UAE itinerary in under ten seconds, and the plan can be argued with in plain English afterwards. More importantly, it answers the question the subject is really about — *what is AI actually good for?*

Generative AI is most useful when it is **given structure**: a clear role, real constraints and a defined output. Left open-ended it writes nice paragraphs. Given a role, a currency, a pacing rule and an exact JSON shape, it becomes a reliable part of a working product — and that is the finding I would carry into the next build.

APPENDIX A1

Structure and Styling

How the markup and the design system are written. All code is taken directly from the project files.

Accessible tab panels

studio.html

```
<div class="tabs" role="tablist" aria-label="Output views">
  <button class="tab" type="button" role="tab" id="tab-itinerary"
    aria-controls="panel-itinerary" aria-selected="true">Itinerary</button>
  <button class="tab" type="button" role="tab" id="tab-pdf"
    aria-controls="panel-pdf" aria-selected="false" tabindex="-1">PDF</button>
  <button class="tab" type="button" role="tab" id="tab-map"
    aria-controls="panel-map" aria-selected="false" tabindex="-1">Map</button>
</div>

<div class="panel-wrap" role="tabpanel" id="panel-itinerary"
  aria-labelledby="tab-itinerary" tabindex="0">
  <div class="scroller" id="itineraryOut"></div>
</div>
```

Method — ARIA tabs. `role`, `aria-controls` and `aria-selected` tell a screen reader that these three buttons switch one region, and the hidden attribute on inactive panels keeps them out of the reading order.

The design system as custom properties

css/tokens.css

```
:root {
  /* Core four */
  --color-bg: #161826;          /* deep navy — the night sky */
  --color-surface: #232532;      /* cards and panels */
  --color-text: #e9e9ed;         /* warm off-white */
  --color-accent: #9184d9;       /* purple — reserved for AI */

  /* Sand ramp — the UAE half of the palette */
  --sand-300: #e0c184;
  --sand-500: #c9a227;          /* gold — heritage and place */

  /* Vertical rhythm: every spacing value is a multiple of these */
  --leading: 28px;
  --half: 14px;
}
```

Method — one source of truth. A token lives here only if its value is identical on both pages, so the two stylesheets can never drift apart. Page-specific values are declared at the top of that page's own stylesheet instead.

Painting a slider's filled track from JavaScript

css/studio.css + js/studio.js

```
input[type='range']::-webkit-slider-runnable-track {
  background: linear-gradient(90deg,
    var(--color-accent) var(--fill, 50%), var(--neutral-700) var(--fill, 50%));
}

function paintRange(input) {
  var pct = ((input.value - input.min) / (input.max - input.min)) * 100;
  input.style.setProperty('--fill', pct + '%'); // CSS does the drawing
}
```

Method — let CSS do the work. JavaScript writes one number; the gradient stop does the painting. No extra element and no layout work while the slider is dragged.

APPENDIX A2

Calling the AI

Prompt engineering, the HTTP request to Google Gemini, and the JSON contract that makes the reply usable.

Stage 02 — building the prompt

js/studio.js · buildPrompt()

```
lines.push('You are an expert United Arab Emirates travel planner.');
```

```
lines.push('- Destination: ' + cfg.destination);
```

```
lines.push('- Duration: ' + cfg.days + ' days');
```

```
lines.push('- Budget per person: ' + aed(cfg.budget) + ' (' + tier.name + ')');
```

```
if (notes.length) {
```

```
  lines.push('TRAVELLER NOTES, most recent last — obey these above all:');
```

```
  notes.forEach(function (note, i) { lines.push((i + 1) + '. ' + note); });
```

```
}
```

```
lines.push('RULES');
```

```
lines.push('- Group each day geographically to minimise travel. Name real UAE places.');
```

```
lines.push('- Respect UAE realities: summer midday heat, Friday prayer timings,');
```

```
lines.push('  modest dress at mosques, Ramadan hours, drive times between emirates.');
```

```
lines.push('- 4 to 6 stops per day, each with a time window, a reason and an AED cost.');
```

```
lines.push('Reply with ONLY valid JSON. No markdown fences. Exactly this shape:');
```

```
lines.push('{ "title": "", "summary": "", "days": [{ "day": 1, "theme": "", "stops": ' +
```

```
  ' [ { "time": "09:00 - 11:00", "title": "", "detail": "", "cost": "AED 120" } ] }, ' +
```

```
  ' "tips": [ "" ], "estimatedTotal": "AED 5,400" }');
```

Method — prompt engineering. The role, currency, pacing rules and exact output schema are all fixed in the prompt. This is what turns a conversational model into a component the rest of the program can rely on.

The request body

js/studio.js · generate()

```
var body = {
```

```
  contents: [{ role: 'user', parts: [{ text: buildPrompt(cfg, state.refinements) }] }],
```

```
  generationConfig: {
```

```
    temperature: 0.9, // high enough to vary between runs
```

```
    responseType: 'application/json' // ask the API itself for JSON
```

```
  }
```

```
};
```

The API call itself

js/studio.js · callModel()

```
var API = 'https://generativelanguage.googleapis.com/v1beta';
```

```
async function callModel(key, model, body) {
```

```
  var res = await fetch(
```

```
    API + '/models/' + encodeURIComponent(model) +
```

```
    ':generateContent?key=' + encodeURIComponent(key),
```

```
    {
```

```
      method: 'POST',
```

```
      headers: { 'Content-Type': 'application/json' },
```

```
      body: JSON.stringify(body)
```

```
    }
```

```
  );
```

```
  var data = null;
```

```
  try { data = await res.json(); } catch (err) { data = null; }
```

```
  if (!res.ok) throw httpError(res, data); // surface Google's own message
```

```
  var parts = data.candidates[0].content.parts;
```

```
  var text = parts.map(function (p) { return p.text || ''; }).join('').trim();
```

```
  if (!text) throw new Error('Empty response from ' + model);
```

```
  return text;
```

```
}
```

Method — a direct browser call. There is no backend, so the request goes from the user's own browser straight to Google. The key is stored only in that browser's local storage and is never committed.

APPENDIX A3

Making It Reliable

Three defences: surviving retired models, surviving malformed replies, and surviving the model's own text.

Walking a fallback chain of models

js/studio.js · callWithFallback()

```
var RETRYABLE = { 404: true, 429: true, 500: true, 503: true, 504: true };

async function callWithFallback(key, chain, body, onHop) {
  var firstError = null;

  for (var i = 0; i < chain.length; i++) {
    try {
      var text = await callModel(key, chain[i], body);
      return { text: text, model: chain[i], fellBack: i > 0 };
    } catch (err) {
      // Keep the FIRST failure, never the last: it came from the model the
      // user actually chose, so it is the one worth reporting.
      if (!firstError) firstError = err;

      if (!RETRYABLE[err.status] || i === chain.length - 1) throw firstError;

      onHop(chain[i], chain[i + 1]); // tell the user in the chat
      await sleep(400);
    }
  }
}
```

Method — never hard-code model names. Before this runs, the app asks GET /models which models the key can actually call and filters its preference list against the answer. An earlier version had fixed model IDs; Google retired them and every request returned 404.

Parsing a reply that may not be clean JSON

js/studio.js · parseJSON()

```
function parseJSON(text) {
  var cleaned = String(text)
    .replace(/^s*````(?:json)?s*/i, '') // strip an opening markdown fence
    .replace(/s*````s*$/, '') // and a closing one
    .trim();

  try { return JSON.parse(cleaned); } catch (err) { /* fall through */ }

  // Second attempt: the widest {...} substring, for when the model wrote a
  // sentence before the JSON.
  var first = cleaned.indexOf('{');
  var last = cleaned.lastIndexOf('}');
  if (first !== -1 && last > first) {
    try { return JSON.parse(cleaned.slice(first, last + 1)); } catch (err) {}
  }

  return null; // caller renders the prose safely instead of failing
}
```

Rendering model output safely

js/studio.js · el()

```
// Every element in the output is built this way and never with innerHTML,
// because the content comes from a language model. A model that emits
// "<img onerror=...>" as a stop title must produce a badly-named stop,
// not a running script.
function el(tag, className, text) {
  var node = document.createElement(tag);
  if (className) node.className = className;
  if (text !== undefined && text !== null) node.textContent = String(text);
  return node;
}
```

Method — treat AI output as untrusted input. This is the same principle as escaping anything typed by a user: the text is displayed, never executed.

APPENDIX A4

Offline Planner and PDF Export

The parts that work with no internet at all, and the document the user walks away with.

A seeded random number generator

js/studio.js · seeded()

```
function seeded(seed) {
  // Knuth multiplicative hash spreads small consecutive seeds (1, 2, 3...)
  // across the whole 32-bit range, then a linear congruential generator
  // steps through it.
  var s = (Math.imul(seed ^ 0x9e3779b9, 2654435761) >>> 0) || 1;
  return function () {
    s = (Math.imul(s, 1664525) + 1013904223) >>> 0;
    return s / 4294967296;
  };
}

function shuffled(list, rnd) {          // Fisher-Yates
  var a = list.slice();
  for (var i = a.length - 1; i > 0; i--) {
    var j = Math.floor(rnd() * (i + 1));
    var tmp = a[i]; a[i] = a[j]; a[j] = tmp;
  }
  return a;
}
```

Method — seeded, not random. The planner is a pure function, so pressing Generate twice used to return an identical plan. A run counter now feeds this generator, and because it is *seeded* rather than `Math.random()`, one seed always produces one ordering — the behaviour stays reproducible and testable.

Reacting to the chat without a model

js/studio.js · offlinePlan()

```
var RX_LIGHTER = /toddler|kid|child|family|slow|light|relax|tired|elderly/i;
var RX_CHEAPER = /cheap|budget|less|afford|save/i;
var RX_FOOD    = /food|eat|dining|restaurant|cuisine/i;

var lighter = RX_LIGHTER.test(joined);    // fewer stops per day
var cheaper = RX_CHEAPER.test(joined);    // bias the pool toward free stops
var perDayStops = perDay > 2000 ? 6 : (perDay > 700 ? 5 : 4);
if (lighter) perDayStops = 3;
```

Method — and its honest limit. This is pattern matching, not understanding, and the app says so in the chat. Only the Gemini path genuinely comprehends a note like “make day 4 lighter, we have a toddler”.

Typesetting the itinerary as a PDF

js/studio.js · buildPDF()

```
var doc = new window.jspdf.jsPDF({ unit: 'pt', format: 'a4' });

doc.setFillColor(22, 24, 38);           // the navy header band
doc.rect(0, 0, pageW, 116, 'F');
doc.setTextColor(145, 132, 217);       // the plan title in purple
doc.text(plan.title, margin, 94);

function need(height) {                 // page-break awareness
  if (y + height ≤ pageH - 56) return;
  doc.addPage();
  y = margin + 18;
}

// Preview the exact file that will download, then release the old one.
if (state.pdfUrl) URL.revokeObjectURL(state.pdfUrl);
state.pdfUrl = URL.createObjectURL(doc.output('blob'));
pdfFrame.src = state.pdfUrl + '#view=FitH';
```

Method — real text, not a screenshot. The document is laid out line by line, so the output is selectable and searchable. `need()` asks for room before drawing a block, so a day heading never lands alone at the foot of a page.